Chapter 1: Make failure explicit

An operation can fail even when its inputs look reasonable. Represent expected failures as typed errors. Add context at the application boundary so the person running the program can tell what failed and what to do next.

Avoid returning an empty success when an operation never completed. Tests should exercise the failure path as well as the happy path. A clear error is more useful than a plausible result.

Chapter 2: Make retries safe

A retry should not create the same effect twice. Use an idempotency key to recognize a repeated request before applying its side effect. Store the key and the result together so a later retry can return the original result.

For example, a job submission can carry a stable request identifier. If the first response is lost, submitting the same identifier should find the existing job. Test the duplicate request explicitly; a retry loop alone does not make an operation safe.

Chapter 3: Keep evidence close

A useful engineering note includes a claim, the reason it matters, and a source that another reader can inspect. Distinguish your interpretation from the source's exact words. A page reference lets a teammate verify the reasoning.

When a source changes, review notes derived from it. A saved lesson is a working interpretation, not a substitute for testing the current implementation. Keep examples small enough that a reader can reproduce them.